

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования

«Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

Отчёт по лабораторной работе № 1

по дисциплине «Расширенный курс по бэкенд-разработке в среде Node.js»

Реализация REST API сервиса «Lecter's Recipes»

Вариант 5

Выполнил: студент группы БР1.1

Кошкарев Кирилл Павлович

Проверил:

Санкт-Петербург, 2026

1. Цель работы

На основе схемы базы данных из ДЗ1 и спецификации API из ДЗ2 реализовать полностью работающее REST API сервиса обмена рецептами: модели, представления и контроллеры.

2. Используемый стек

Таблица 1 — Стек технологий приложения

Технология	Версия	Роль в проекте
Node.js	22	среда выполнения
TypeScript	5.7	статическая типизация, строгий режим
Express	4.21	HTTP-слой, маршрутизация, middleware
TypeORM	0.3	ORM, сущности-декораторы, построитель запросов
PostgreSQL	17	СУБД
jsonwebtoken, bcryptjs	9.0, 2.4	выпуск и проверка JWT, хеширование паролей
Zod	3.24	валидация входных данных по схемам
swagger-ui-express, yamlsjs	5.0, 0.3	публикация спецификации из ДЗ2
helmet, cors, morgan	8.0, 2.8, 1.10	заголовки безопасности, CORS, журнал запросов

За основу взята типовая структура boilerplate'a express + typeorm: разделение на модели, контроллеры, сервисы и маршруты, конфигурация через переменные окружения, единый обработчик ошибок.

3. Структура проекта

lecters-recipes-api/	
├─ src/	
│ └─ models/	13 сущностей TypeORM (14-я таблица recipe_tags – таблица связи)
│ └─ views/	представления: сериализация сущностей в DTO ответов
│ └─ controllers/	контроллеры: разбор запроса → вызов сервиса → HTTP-ответ
│ └─ services/	бизнес-логика: auth, user, recipe, social, dictionary
│ └─ routes/	маршруты и привязка middleware
│ └─ middlewares/	аутентификация, валидация, обработчик ошибок
│ └─ dto/	zod-схемы входных данных
│ └─ config/	окружение, DataSource, тема Swagger UI
│ └─ utils/	ошибки, пагинация, JWT, пароли, транслитерация слогов
│ └─ app.ts	сборка express-приложения
│ └─ server.ts	точка входа
│ └─ seed.ts	демонстрационные данные
├─ public/	витрина – статический клиент поверх API
├─ docs/openapi.yaml	спецификация из ДЗ2
└─ scripts/	создание БД, приёмочный прогон, валидация OpenAPI

Листинг 1 — структура исходного кода

4. Модели

Все 14 таблиц ДЗ1 описаны сущностями TypeORM. Типы столбцов указаны явно, поэтому приложение не зависит от метаданных декораторов и одинаково работает при запуске через tsx и после компиляции. Связь «рецепт — теги» описана декоратором `@JoinTable` и создаёт таблицу `recipe_tags`.

```
@Entity('recipes')
export class Recipe {
  @PrimaryGeneratedColumn({ type: 'bigint' }) id: string;

  @Index() @Column({ type: 'bigint' }) authorId: string;

  @ManyToOne(() => User, (user) => user.recipes, { onDelete: 'CASCADE' })
  @JoinColumn({ name: 'authorId' }) author: User;

  @Index({ unique: true }) @Column({ type: 'varchar', length: 180 }) slug: string;

  @Index() @Column({ type: 'enum', enum: RecipeStatus, default: RecipeStatus.DRAFT })
  status: RecipeStatus;

  @ManyToMany(() => Tag, (tag) => tag.recipes)
  @JoinTable({ name: 'recipe_tags',
    joinColumn: { name: 'recipeId', referencedColumnName: 'id' },
    inverseJoinColumn: { name: 'tagId', referencedColumnName: 'id' } })
  tags: Tag[];

  @OneToMany(() => RecipeIngredient, (ri) => ri.recipe, { cascade: true })
  ingredients: RecipeIngredient[];
}
```

Листинг 2 — фрагмент модели Recipe

Поле `passwordHash` сущности `User` объявлено с опцией `select: false` — оно не попадает в выборку по умолчанию и запрашивается явно только при проверке пароля.

5. Представления

Представления — чистые функции, переводящие сущность в структуру ответа API. Они решают три задачи: скрывают служебные поля, приводят типы (TypeORM возвращает `bigint` и `numeric` строками — наружу уходят числа) и гарантируют, что форма ответа совпадает со схемой из ДЗ2 независимо от места вызова.

```

export const recipeCardView = (recipe: Recipe) => ({
  id: id(recipe.id),
  title: recipe.title,
  difficulty: recipe.difficulty,
  cookTimeMinutes: recipe.cookTimeMinutes,
  likesCount: recipe.likesCount,
  author: recipe.author ? publicUserView(recipe.author) : null,
  category: categoryView(recipe.category),
  tags: (recipe.tags ?? []).map(tagView),
});

export const recipeView = (recipe: Recipe, viewer: RecipeViewerState = {}) => ({
  ...recipeCardView(recipe),
  ingredients: [...(recipe.ingredients ?? [])]
    .sort((a, b) => a.position - b.position).map(recipeIngredientView),
  steps: [...(recipe.steps ?? [])]
    .sort((a, b) => a.stepNumber - b.stepNumber).map(recipeStepView),
  isLiked: viewer.isLiked ?? false,
  isFavorite: viewer.isFavorite ?? false,
});

```

Листинг 3 — представления карточки и страницы рецепта

6. Контроллеры и маршруты

Контроллер не содержит бизнес-логики: он извлекает данные из запроса, вызывает сервис и передаёт результат в представление. Общие проверки вынесены в `middleware`, поэтому маршрут читается как декларация требований к запросу.

```

recipeRouter.post(
 ('/:id/publish',
  authenticate, // требуется действующий access-токен
  validate(idParamSchema, 'params'), // id — положительное целое
  asyncHandler(recipeController.publish),
);

// контроллер
async publish(req: Request, res: Response): Promise<void> {
  const recipe = await loadOwned(req); // 404 если нет, 403 если не автор
  const published = await recipeService.publish(recipe); // 422 если нет шагов
  res.json(recipeView(published, await recipeService.viewerState(published.id, req.user?.id)));
}

```

Листинг 4 — маршрут и контроллер публикации рецепта

7. Аутентификация и разграничение доступа

Пароль хранится в виде `bcrypt`-хеша. При входе выдаётся пара токенов: `access` на час и `refresh` на неделю; тип токена записан в полезной нагрузке, поэтому `refresh`-токен нельзя использовать вместо `access`. Реализованы три `middleware`:

- **authenticate** — требует действующий токен, иначе 401;

- **optionalAuth** — заполняет req.user при наличии токена, но не требует его; нужен там, где ответ зависит от зрителя (черновики видны только автору, isLiked вычисляется для текущего пользователя);
- **requireAdmin** — ограничивает изменение справочников ролью администратора.

Проверка «изменять может только автор» выполняется в сервисном слое: контроллер загружает рецепт и вызывает assertCanModify, который возбуждает 403 для чужого рецепта — при этом администратор проходит проверку.

8. Валидация и обработка ошибок

Каждый эндпоинт объявляет zod-схему тела, параметров пути или строки запроса. При нарушении middleware validate формирует ошибку 400 со списком полей — ровно в том формате, который описан в Д32. Прикладные ошибки выражены классом AppError со статусом и кодом; единый обработчик дополнительно распознаёт ошибки драйвера PostgreSQL: нарушение уникальности (23505) превращается в 409, нарушение внешнего ключа (23503) — в 404.

9. Поиск и фильтрация

Основной сценарий витрины реализован построителем запросов TypeORM. Фильтры комбинируются по «И»; фильтр по ингредиентам требует наличия *всех* перечисленных позиций, что выражено подзапросом с подсчётом совпадений.

```
if (query.ingredientId?.length) {
  qb.andWhere(
    `(SELECT COUNT(DISTINCT ri."ingredientId") FROM recipe_ingredients ri
      WHERE ri."recipeId" = recipe.id AND ri."ingredientId" IN (...ingredientIds))
    = :ingredientCount`,
    { ingredientIds: query.ingredientId, ingredientCount: query.ingredientId.length },
  );
}
if (query.tag?.length) {
  qb.andWhere(
    `EXISTS (SELECT 1 FROM recipe_tags rt JOIN tags t ON t.id = rt."tagId"
      WHERE rt."recipeId" = recipe.id AND t.slug IN (...tagSlugs))`,
    { tagSlugs: query.tag },
  );
}
```

Листинг 5 — фильтрация по составу и тегам

Отдельного внимания потребовала локаль базы данных. При создании базы с параметрами по умолчанию (`LC_COLLATE = C`) оператор `ILIKE` не сворачивает регистр кириллицы, и поиск по названию рецепта не находил совпадений с заглавной буквы. Проблема обнаружена приёмочным прогоном; в проект добавлен скрипт создания базы с локалью `en_US.UTF-8`, а в `docker-compose` — соответствующий параметр `POSTGRES_INITDB_ARGS`.

10. Документация и витрина

Приложение отдаёт спецификацию из Д32 по адресам `/openapi.yaml` и `/openapi.json` и публикует Swagger UI на `/docs` с оформлением в палитре проекта. Дополнительно реализована витрина — статический клиент на `/`, работающий поверх того же API: поиск с фильтрами, карточка рецепта, лайки, избранное, комментарии, подписки и форма создания рецепта.

Изображения рецептов приложение отдаёт само: файлы лежат в каталоге `public/images` и адресуются путём от корня сайта (`/images/osso-buco.jpg`). Это избавляет от зависимости от внешних хостов и от привязки к конкретному порту или домену. Валидация ссылок на медиа принимает и абсолютный `http(s)-URL`, и такой путь, что отражено в спецификации форматом `uri-reference`. Фотографии блюд взяты с Викисклада по свободным лицензиям, авторы и условия перечислены в файле `public/images/CREDITS.md`; аватары пользователей — сгенерированные монограммы.

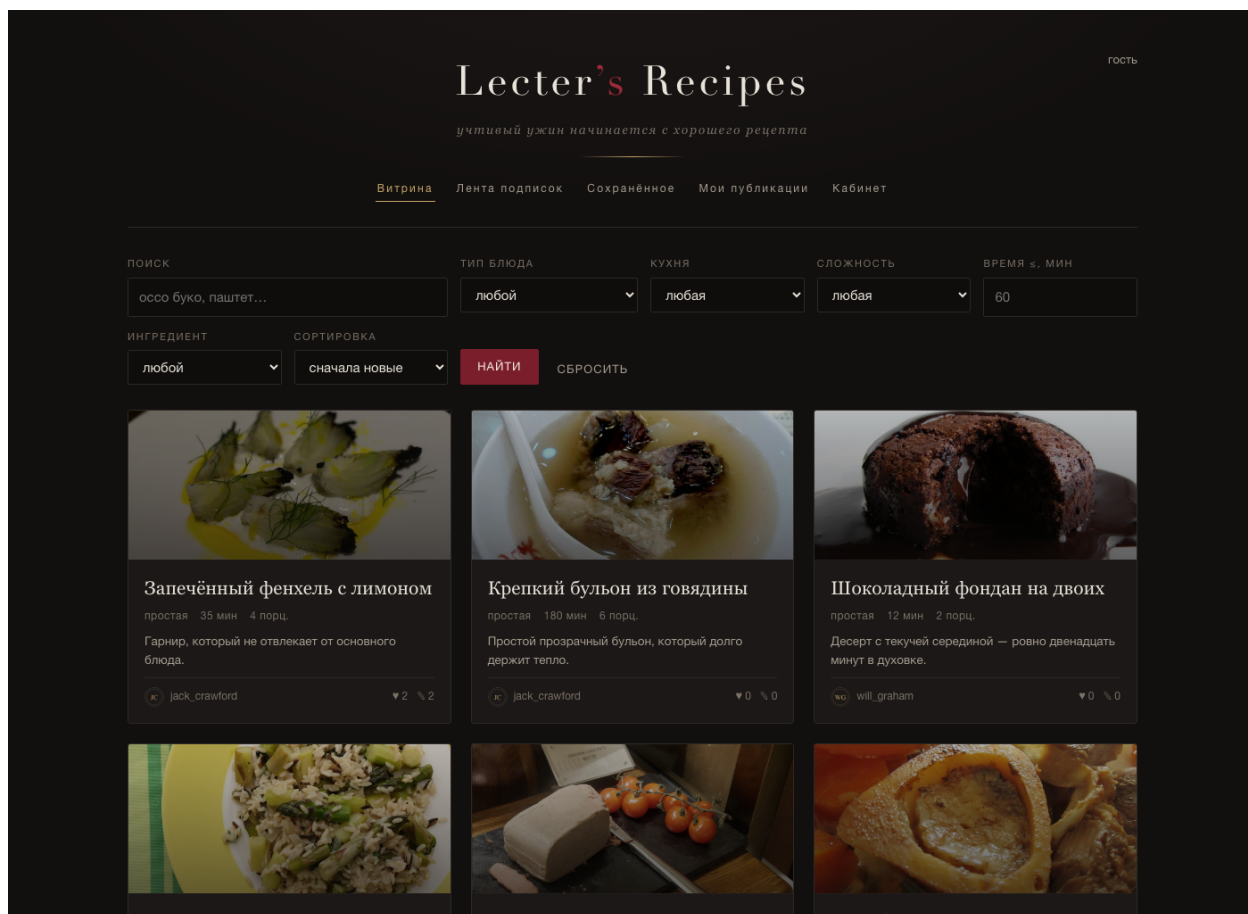


Рисунок 6 — витрина «Lecter's Recipes»: поиск с фильтрами и карточки рецептов

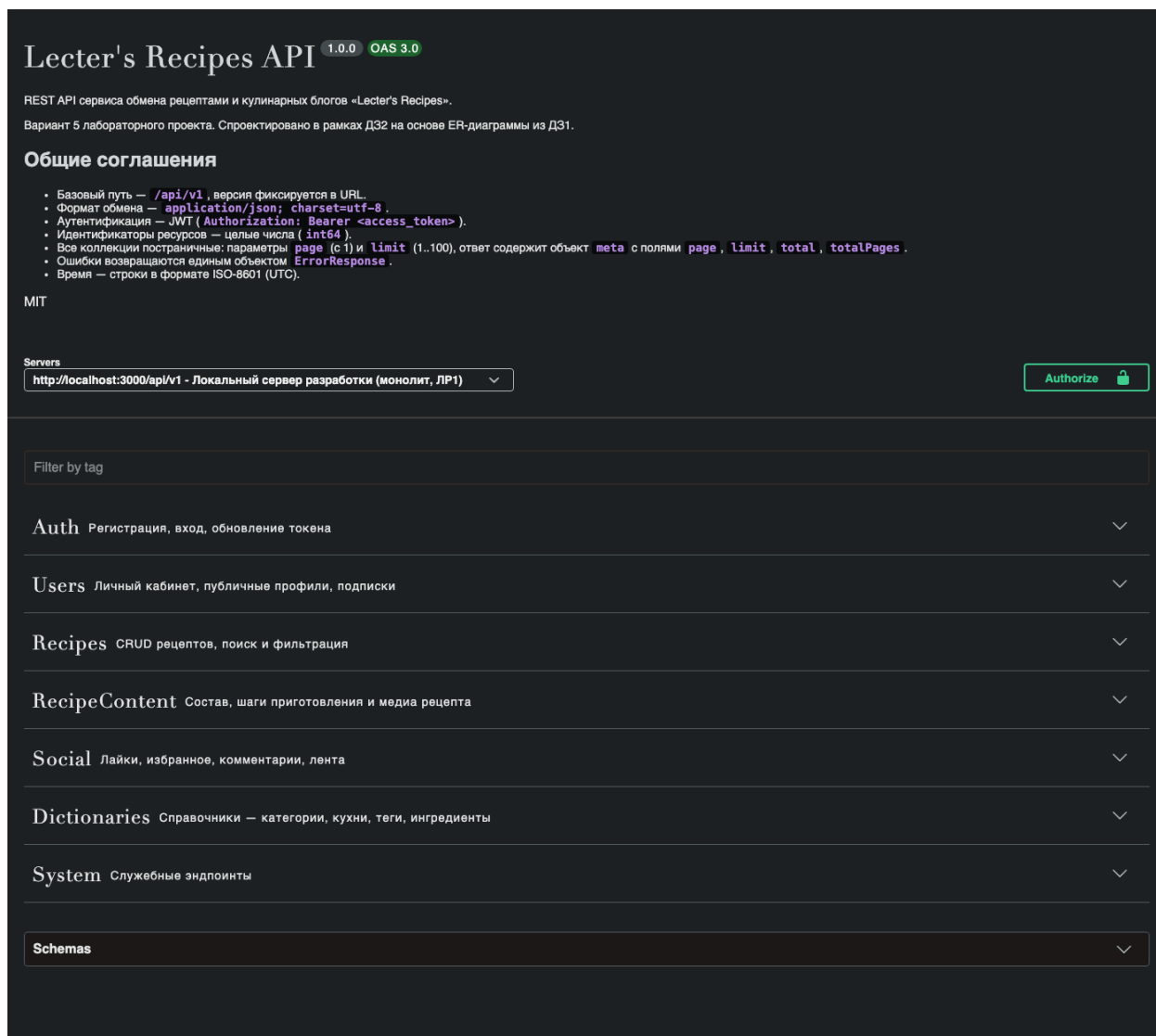


Рисунок 7 — Swagger UI со спецификацией из Д32

11. Проверка работоспособности

Для приёмки написан сценарий `scripts/smoke.ts`, проходящий по всем требованиям Д32: регистрация, вход, обновление токена, личный кабинет, справочники, жизненный цикл рецепта, фильтрация, социальные функции, права доступа и коды ошибок. Прогон выполняется против запущенного приложения.

```

$ npm run smoke
SMOKE: Lecter's Recipes API @ http://localhost:3000

1. Служебные эндпоинты      □ GET /health → status=ok
2. Регистрация и вход      □ повторный email → 409 CONFLICT
                             □ ошибка содержит details по полям
3. Личный кабинет          □ GET /users/me с мусорным токеном → 401
4. Справочники             □ POST /categories обычным пользователем → 403
5. Жизненный цикл рецепта  □ черновик не виден анониму → 404
                             □ публикация без шагов → 422 UNPROCESSABLE
6. Поиск и фильтрация      □ фильтр по ингредиенту
                             □ в общей выдаче нет черновиков
7. Социальные функции      □ повторный лайк идемпотентен
                             □ GET /users/me/feed содержит рецепт автора
8. Права доступа и ошибки  □ изменение чужого рецепта → 403
9. Удаление                 □ удалённый рецепт → 404

ИТОГО: 70 успешно, 0 провалено

```

Листинг 8 — результат приёмочного прогона (сокращённый вывод)

12. Выводы

Реализовано полностью работающее REST API по варианту 5: 14 таблиц ДЗ1 отражены в моделях TypeORM, 44 операции ДЗ2 — в контроллерах и маршрутах, все ответы проходят через слой представлений. Работоспособность подтверждена приёмочным сценарием из 70 проверок, включая негативные: 400, 401, 403, 404, 409 и 422.

Разделение на модели, представления, контроллеры и сервисы окупилось в следующих работах: при переходе к микросервисам (ЛР2) сервисный слой и представления переносились между приложениями практически без изменений.